

9/15 논문 - Attention is all you need

1. Introduction

[NIPS-2017-attention-is-all-you-need-Paper.pdf](#)

lstm 과 gru가 시퀀스 모델링에서 주류였음

- GRU : rnn 기반 모델 , cell state (기억을 이어가는 상태) 없이 update(이전 기억을 얼마나 유지, 새 정보로 얼마나 갱신할지 조절) , reset(새로운 기억 후보를 만들 때 이전 기억을 얼마나 참고할지 조절) 이 두개의 게이트를 사용해 hidden state 하나로 기억을 관리
- 시퀀스 모델링: 순서 있는 데이터의 패턴 배우기
- → 예시 : 베이지안을 이용한 다음 단어 예측
- 시퀀스 변환: 입력 시퀀스를 바탕으로 출력 시퀀스 만들기
- → 예시 : 기계 번역

encoder (입력 → 표현)- decoder (표현 → 출력) 연구가 지속

순환 모델 : 입력과 출력 시퀀스의 기호 위치에 따라 계산을 나누어 수행

- $h_1 = f(h_0, \text{나는})$ 이런 식으로 각 위치에 대응하는 계산 단계가 생김

현재 상태는 현재 입력 + 이전 상태에 의존 ⇒ 문장 속 위치 순서가 계산 순서로 이어짐

병렬 처리

1. 한 문장 내부 : 다음 위치는 이전 hidden state를 기다려야 함
2. 여러 문장 사이: 서로 다른 문장들은 배치로 묶어서 동시에 처리할 수 있음

⇒ 문장이 길수록 순차적으로 거쳐야할 단계는 증가, 여러 문장을 한번에 처리해서 보완하는데도 한계가 생김

*서로 의존하는 시퀀스들의 계산을 동시에 수행할 수 없음

두가지 기법을 통해 계산 효율 개선

1. factorization tricks 분해 기법 : 큰 계산을 더 작은 연산의 조합으로 분해해서 계산 cost 줄임
2. conditional computation 조건부 계산 : 입력에 따라 모델의 일부만 선택해서 계산하는 방식

순차적인 계산은 제약이 남아있음 : 앞 단계가 끝나기를 기다려야 하는 구조 자체는 해결 x

Attention method : 현재 처리하는 위치에서 필요한 다른 위치의 정보를 가중치를 주어 참조하게 함. → 멀리 있는 정보도 활용 가능 - 기존 모델에서는 attention을 rnn에 추가해서 사용, rnn이 순서대로 문장을 처리해서 hidden state 생성 하고 attention이 필요한 정보를 찾아서 참조

그래서 rnn의 순환구조를 제거하고 attention으로 요소 사이의 관계를 계산하겠다~

transformer는 병렬처리를 많이 가능하게 함

논문이 다루는 분야

자연어 처리(NLP)의 Sequence transduction과 Sequence modeling을 다루며 주요 실험 과제는 입력 문장을 받아 다른 언어의 문장을 생성하는 기계번역이다.

논문이 쓰여질 당시에는 LSTM과 GRU같은 RNN 계열 모델이 주요 방법으로 사용되어 입력을 표현으로 바꾸는 encoder 와 이를 바탕으로 출력하는 decoder구조가 널리 연구되었다. Attention도 대부분 이러한 RNN 기반 모델과 결합되어 일부 필요한 정보만 참조되는 형식으로 사용되었다.

이 논문은 RNN의 순환 구조 없이 Attention을 중심으로 문장을 처리하는 Transformer를 제안하였다.

해당 task에서 기존 연구 한계점

RNN은 현재 hidden state를 계산할 때 현재 입력과 이전의 hidden state를 함께 사용한다 :

$$(h_t = f(h_{t-1}, x_t))$$

따라서 문장 전체가 주어져도 이전 위치의 계산이 끝나야만 다음 위치를 계산한다. 이는 문장 속 위치 순서가 계산 순서로 이어지므로 한 문장 내부의 위치들을 병렬로 처리하는데 어려움을 유발한다.

서로 다른 문장들을 배치로 묶어 동시에 처리할 수는 있지만, GPU 메모리에 한계가 있기에 배치 크기를 계속 늘릴 수는 없다. 또한 문장이 길어지면 순차적으로 거쳐야 할 단계와 저장할 중간 상태도 늘어나기에 문제가 커진다.

기존 연구들은 분해 기법 혹은 조건부 계산을 이용해 계산 효율을 개선했다. 혹은 Attention을 RNN에 결합시켜 멀리 떨어진 위치의 정보 참조가 가능하게 했다. 하지만 이는 RNN의 구조를 유지하였기에 앞 단계의 계산을 기다려야한다는 근본적인 제약은 남아 있었다.

논문의 contributions

1. 순환 구조를 사용하지 않는 Transformer를 제안
 - RNN없이 Attention으로 Sequence 내 위치 사이의 관계와 입력 및 출력 사이의 관계를 모델링하도록함
1. Sequence 처리의 학습 병렬화 높임
 - 이전 위치의 hidden state를 순차적으로 계산해야 하는 구조를 제거, 여러 위치의 표현을 병렬로 계산할 수 있게 하였다.
1. 번역 품질과 학습 효율을 동시에 제시
 - P100 GPU 8개로 12시간 학습한 조건에서도 논문 작성 당시의 최고 수준 번역 품질에 도달할 수 있다고 제시함. 병렬화를 늘리면서 좋은 번역 성능을 얻을 수 있다는 것이 논문의 핵심

2. Related Work



Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

cnn을 기반으로 해결하기로 함
각 위치에서 주변 단어 3개를 보는
convolution을 적용하게 된다면 각 위치
의 계산에 필요한 입력은 이미 주어져 있으
니 여러 위치를 동시에 계산할 수 있음

cnn으로 병렬화는 가능했지만 위치가 증
가하면 여러 층을 거쳐야함 그래서

convS2S :: 거리가 멀어지는 만큼 단계
수도 비례해서 선형 증가

ByteNet: 간격을 벌려 정보를 보는
dilated convolution을 사용해 범위를
빠르게 넓힘 그래서 로그 증가의 속도로 단
계수가 증가함

*연산 단계 : 두 위치의 정보가 연결되기
까지 거치는 경로의 길이

⇒ 멀리 떨어진 위치 사이의 의존관계를
학습하기가 더 어려워짐.

transformer에서는 두 위치의 정보를 연결하는데 필요한 연산 단계를 줄임

Self attention에서는 한 위치가 멀리 있는 다른 위치도 직접 참조 가능

두 위치를 연결하는 경로인 길이가 거리에 따라 $\propto$ 늘어나지 않는다는거임

그래서 참조한 정보들을 가중합으로 섞어 하나의 표현을 만드는데 이때 여러 위치의 정보가 하나로 섞이면서 각 위치가 가진 서로 다른 특징이 뚜렷하게 유지되지 않을 수 있다.

이를 보완하는것이 multi head attention이다. 여러 attention이 서로 다른 방식으로 정보를 참조, 그 결과를 함께 활용하게 하는것

⇒ 먼 위치도 짧은 경로로 연결할 수 있지만 정보의 섞이면서 구별이 흐려질 수도 있기에 여러 개의 attention을 사용한다.

self attention : 같은 sequence 안의 정보를 서로 참조한다.

기존 Self-Attention 연구와 Transformer의 차별점

Self-Attention 자체가 이 논문에서 처음 나온 것은 아니다. 기존에도 독해, 생성형 요약, 문장 간 함의 판단, 특정 과제에 종속되지 않는 문장 표현 학습 등에 사용되었다. End-to-End Memory Network도 Attention을 반복적으로 적용해 정보를 읽는 방식을 사용했고, 간단한 언어로 된 질의응답과 언어 모델링에서 좋은 결과를 보였다. 여기서 Attention의 반복은 문장의 각 위치를 따라 진행하는 RNN의 순환과 구분된다. (원문 §2)

Transformer의 차별점은 Attention을 보조 기능으로 추가하는 데서 그치지 않았다는 것이다. 저자들은 자신들이 아는 범위에서, 시퀀스 위치를 따라 계산하는 RNN이나 convolution 없이 Self-Attention으로 입력과 출력 표현을 계산하는 최초의 시퀀스 변환 모델이라고 설명한다.

따라서 Related Work의 흐름은 다음과 같다. 기존 CNN 기반 연구들도 병렬화를 추구했지만 먼 위치를 연결하는 경로가 길어졌다. Transformer는 Self-Attention으로 그 경로를 짧게 만들고, 정보를 가중합하면서 서로 다른 특징이 섞이는 문제는 Multi-Head Attention으로 완화한다. Attention의 존재 자체보다, 그것을 중심으로 시퀀스 처리 구조를 다시 설계했다는 것이 핵심이다.

3. 제안 방법론



Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

1. 전체 구조: Encoder-Decoder

Transformer는 입력을 해석하는 encoder와 출력을 생성하는 decoder라는 틀을 유지한다. 다만 위치 사이의 정보를 전달하는 RNN과 CNN을 Attention으로 대체한다. Encoder는 입력 문장을 벡터 하나로 압축하는 것이 아니라, 각 입력 위치에 대응하는 문맥 표현을 만든다. Decoder는 이 표현과 앞서 나온 출력 토큰을 이용해 다음 토큰을 예측한다. (원문 §3, Figure 1)

Base 모델은 encoder와 decoder를 각각 6개 층으로 쌓고, 표현 차원 d_{model} 은 512로 설정한다. 같은 구조의 층을 반복한다는 뜻이며, 층 사이의 가중치를 모두 공유한다는 뜻은 아니다.

- Encoder의 각 층: Multi-Head Self-Attention → Feed-Forward Network.
- Decoder의 각 층: Masked Multi-Head Self-Attention → Encoder-Decoder Attention → Feed-Forward Network.
- 각 하위 층에는 residual connection과 Layer Normalization을 적용한다.

기본 형태는 다음과 같다. 실제 학습에서는 하위 층 출력에 dropout을 적용한 뒤 입력과 더한다.

출력 = $\text{LayerNorm}(x + \text{Sublayer}(x))$

2. Scaled Dot-Product Attention

Attention은 query와 key를 비교해 가중치를 구하고, 그 가중치로 value들을 합치는 연산이다. (원문 §3.2.1, Figure 2)

- Query(Q): 현재 위치에서 어떤 정보를 참조할지 판단하는 벡터.
- Key(K): 각 참조 대상이 query와 얼마나 관련 있는지 비교하는 벡터.
- Value(V): 가중치를 적용해 실제로 모아 올 정보 벡터.

이 역할들은 이해를 위한 설명이며, 실제 벡터는 학습 가능한 선형 변환으로 만들어진다.

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$

먼저 Q와 K의 내적으로 관련성 점수를 구한다. 이를 $\sqrt{d_k}$ 로 나누고, 각 query에 대해 softmax를 적용해 가중치의 합이 1이 되도록 만든다. 마지막으로 그 가중치를 V에 곱해 각 위치의 새 표현을 계산한다.

$\sqrt{d_k}$ 로 나누는 이유도 중요하다. 차원이 커지면 내적 값의 크기가 커져 softmax가 한쪽으로 치우치고 gradient가 매우 작아질 수 있다. Scaling은 이 문제를 완화하기 위한 장치다. 또 query들을 행렬로 묶어 계산하므로 여러 위치의 Attention을 행렬 곱으로 함께 처리할 수 있다.

3. Multi-Head Attention

Attention을 한 번만 계산하면 여러 위치의 정보가 하나의 가중합으로 섞인다. Multi-Head Attention은 서로 다른 학습 가능한 선형 변환으로 Q, K, V를 여러 표현 공간에 투영하고, 각 공간에서 Attention을 병렬로 계산한다. 이후 결과를 이어 붙이고 다시 선형 변환한다. (원문 §3.2.2)

```
head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)
```

```
MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O
```

Base 모델은 head 8개를 사용하고, 각 head의 key와 value 차원은 64다. 512차원 Attention을 8번 그대로 반복하는 것이 아니라 각 head의 차원을 줄이므로, 전체 계산 비용은 전체 차원을 사용하는 single-head Attention과 비슷하게 유지된다.

여러 head가 서로 다른 위치와 표현의 특징을 함께 참조할 수 있다는 것이 장점이다. 다만 각 head에 주어·동사 같은 역할을 사람이 미리 지정해 주는 것은 아니다.

4. 세 종류의 Attention과 Masking

사용 위치	Q의 출처	K, V의 출처	역할
Encoder Self-Attention	Encoder의 현재 층 입력	같은 입력	입력 문장 내부의 관계 반영
Decoder Masked Self-Attention	Decoder의 현재 층 입력	같은 입력	미래 위치를 보지 않고 출력 문맥 반영
Encoder-Decoder Attention	Decoder의 앞 하위 층 출력	Encoder의 최종 출력	출력 생성에 필요한 입력 정보 참조

학습할 때는 정답 출력 문장을 한 위치 오른쪽으로 이동시켜 decoder에 넣는다. 그리고 미래 위치의 Attention 점수에 $-\infty$ 를 넣어 softmax 이후 가중치가 0이 되도록 한다. 이 두 장치로 현재 예측이 정답 토큰이나 미래 토큰을 미리 보는 것을 막는다. (원문 §3.1, §3.2.3)

여기서 학습과 생성을 구분해야 한다. 학습에서는 정답 출력들이 이미 주어져 있으므로 mask를 적용한 상태에서 여러 위치를 병렬로 계산할 수 있다. 실제 번역 생성에서는 다음 입력으로 쓸 출력 토큰을 먼저 생성해야 하므로 여전히 한 토큰씩 진행한다. RNN을 제거했다고 생성 과정의 순차성까지 사라지는 것은 아니다.

5. Feed-Forward Network와 출력 확률

Attention 다음에는 각 위치에 동일한 FFN을 독립적으로 적용한다. FFN은 두 선형 변환 사이에 ReLU를 둔 구조다. 같은 층 안에서는 위치별로 파라미터를 공유하지만, 층이 달라지면 파라미터도 다르다. (원문 §3.3–3.4)

```
FFN(x) = max(0, xW_1 + b_1)W_2 + b_2
```

Base 모델에서는 $512 \rightarrow 2048 \rightarrow 512$ 차원으로 변환한다. Attention이 위치 사이의 정보를 모은다면, FFN은 그렇게 얻은 각 위치의 표현을 비선형적으로 변환한다. Decoder의 마지막 출력에는 선형 변환과 softmax를 적용해 다음 토큰의 확률을 구한다.

6. Positional Encoding

순환 구조와 convolution을 제거했으므로 토큰의 순서 정보를 별도로 넣어야 한다. 논문은 encoder와 decoder의 입력 embedding에 같은 차원의 positional encoding을 더한다. (원문 §3.5)

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_{model})})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_{model})})$$

pos는 토큰의 위치이고 i는 차원 쌍의 인덱스다. 차원마다 주기가 다른 sin과 cos를 사용해 위치를 표현한다. 저자들은 이 방식이 상대적인 위치 관계를 배우기 쉽게 만들고, 학습 때보다 긴 시퀀스에도 적용할 가능성이 있다고 설명한다. 다만 긴 길이에 대한 일반화가 실험으로 확인되었다는 뜻은 아니다.

4. 실험 및 결과



Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

- **WMT 2014 영어-독일어:** 약 450만 문장 쌍. BPE를 사용하며 입력·출력에서 공유하는 어휘 크기는 약 37,000이다.
- **WMT 2014 영어-프랑스어:** 약 3,600만 문장 쌍. 약 32,000개의 word-piece 어휘를 사용한다.
- **평가 구분:** 주요 번역 성능 비교인 Table 2는 newstest2014, 구성 요소를 바꾼 Table 3은 영어-독일어 개발 세트 newstest2013 결과다. 두 표의 BLEU를 같은 조건의 수치처럼 직접 비교하면 안 된다. (원문 §5.1, §6)

Baseline

비교 대상은 RNN 계열의 GNMT + RL, CNN 계열의 ByteNet과 ConvS2S, 그리고 MoE 등이다. Table 2에서는 기존 단일 모델과 여러 모델을 결합한 ensemble을 구분해 비교한다.

Transformer 내부에서는 base와 big 모델을 비교하고, 별도로 head 수, key 차원, 층 수, 표현 차원, dropout, label smoothing, 위치 표현 방식을 바꾸는 실험을 수행한다. 기존 모델과의 비교는 전체 번역 성능과 비용을 평가하고, 구성 요소 변경 실험은 각 설계가 결과에 미치는 영향을 살펴보는 역할을 한다.

평가 지표는 BLEU다. 생성한 번역과 정답 번역의 n-gram 일치 등을 이용하는 지표로, 여기서는 높을수록 좋은 결과로 해석한다. BLEU 28.4를 정확도 28.4%로 읽으면 안 된다.

결과

1. 주요 번역 성능 — Table 2

모델	영어 → 독일어 BLEU	영어 → 프랑스어 BLEU
GNMT + RL	24.60	39.92
ConvS2S	25.16	40.46
MoE	26.03	40.56
GNMT + RL Ensemble	26.30	41.16
ConvS2S Ensemble	26.36	41.29
Transformer base	27.30	38.10
Transformer big	28.40	41.00

영어 → 독일어에서는 big 모델이 28.4 BLEU를 기록했다. 비교표의 최고 ensemble인 ConvS2S의 26.36보다 약 2.04 BLEU 높고, base도 27.3으로 기존 비교 모델들을 넘어섰다.

영어 → 프랑스어에서는 big의 41.0이 기존 단일 모델 중 최고였던 MoE의 40.56보다 높았다. 하지만 ConvS2S ensemble의 41.29보다는 낮다. 따라서 두 언어쌍에서 모두 모든 ensemble을 이겼다고 쓰면 부정확하다. Base의 38.1 역시 이 과제에서는 기존 주요 모델보다 낮다.

2. 학습 효율 — §5.2, Table 2

설정	학습 step	P100 8개에서 학습 시간	파라미터 수
Base	100,000	약 12시간	약 6,500만
Big	300,000	약 3.5일	약 2억 1,300만

Introduction의 12시간은 base 설정에 해당한다. 28.4와 41.0 BLEU를 기록한 big은 약 3.5일 학습했다. 학습 시간과 최고 성능을 서로 다른 설정에서 가져와 하나의 모델 결과처럼 묶으면 안 된다.

Table 2의 추정 학습 비용은 base가 3.3×10^{18} FLOPs, big이 2.3×10^{19} FLOPs다. 영어 → 프랑스어에서 기존 최고 단일 모델 MoE의 비용 1.2×10^{20} FLOPs와 비교하면 big은 약 19% 수준의 비용으로 더 높은 BLEU를 얻었다. 이 비용은 학습 시간, GPU 수, GPU의 추정 연산 성능을 곱한 값이므로, 모든 모델을 동일한 하드웨어에서 재실험한 실측 비교는 아니다.

3. Multi-Head Attention — Table 3(A)

Head 수를 바꾸면서 각 head의 차원을 조절해 계산량을 비슷하게 유지했다. 영어 → 독일어 개발 세트에서 head 1개는 24.9 BLEU, 8개는 25.8, 16개도 25.8이었다. 32개에서는

25.4로 다시 낮아졌다.

Single-head가 가장 좋은 설정보다 0.9 BLEU 낮다는 점은 여러 head를 사용하는 설계를 뒷받침한다. 다만 head는 많을수록 무조건 좋은 것이 아니다. 전체 차원을 고정하면 head 수가 늘수록 각 head가 사용하는 차원이 줄어든다는 조건도 함께 봐야 한다.

4. Key 차원과 정규화 — Table 3(B), (D)

Key 차원 d_k 를 base의 64에서 16으로 줄이면 BLEU가 25.8에서 25.1로 떨어졌고, 32에서는 25.4였다. 참조 대상과 query의 관계를 판단하는 표현의 크기도 성능에 영향을 준다.

Dropout을 없애면 BLEU가 25.8에서 24.6으로 낮아졌다. Label smoothing을 없애면 perplexity는 4.92에서 4.67로 개선되지만 BLEU는 25.8에서 25.3으로 낮아졌다. 이 결과는 정규화의 필요성과 함께, perplexity와 최종 번역 품질이 같은 방향으로만 움직이지는 않는다는 점을 보여준다.

5. 위치 정보 — Table 3(E)

Sinusoidal positional encoding 대신 학습 가능한 positional embedding을 사용했을 때 BLEU는 25.8에서 25.7로 거의 같았다. 이 실험은 두 위치 표현 방식의 성능이 비슷하다는 근거다. 위치 정보를 아예 제거한 실험은 아니므로, 위치 정보가 필요 없다는 결론은 낼 수 없다.

종합하면, 실험은 Attention 중심 구조가 번역 품질을 유지·개선하면서 학습 비용을 줄일 수 있다는 주장을 뒷받침한다. 다만 어떤 구성에서도 항상 기존 모델보다 좋다는 뜻은 아니며, 모델 크기와 번역 방향, 비교 대상의 단일·ensemble 여부를 함께 확인해야 한다.

5. 결론 (배운점)

의의

이 논문의 핵심은 문장의 순서를 다루기 위해 반드시 계산도 같은 순서로 진행해야 하는 것은 아니라는 점이다. RNN의 순환 구조를 제거하고 Self-Attention으로 위치 사이의 관계를 계산하면서, 순서 정보는 positional encoding으로 제공한다.

단순히 새로운 구조만 제안한 것이 아니라, 기존 구조와의 계산 경로 비교 및 실제 번역 실험을 함께 제시했다는 점에 의의가 있다. 특히 영어 → 독일어에서는 기존 ensemble보다 높은 성능을 보였고, 영어 → 프랑스어에서도 낮은 추정 학습 비용으로 당시 최고 단일 모델 성능을 달성했다. (원문 §4, §6-7)

한계

- **긴 시퀀스의 비용:** 전체 Self-Attention은 모든 위치 쌍을 비교하므로 총별 계산량이 $O(n^2d)$ 이다. 연결 경로가 짧다는 장점과 별개로, 시퀀스가 길어지면 비용이 커진다. 저자들은 주변 위치만 참조하는 restricted attention을 향후 연구로 제안한다.

- **생성은 여전히 순차적:** 학습 중 위치별 계산은 병렬화하지만, 번역 생성에서는 이전에 생성한 토큰을 바탕으로 다음 토큰을 만든다. 논문도 생성의 순차성을 줄이는 것을 후속 목표로 남긴다.
- **검증 범위와 비교 조건:** 이 NIPS 2017 판본의 주요 실험은 두 기계번역 과제다. 다른 입력 형태나 모든 시퀀스 과제로 결과를 바로 일반화할 수는 없다. 또 학습 비용은 문헌의 시간과 하드웨어 정보에 기반한 추정치라는 점을 고려해야 한다.

배운 점

지금까지 읽으며 정리한 것은 병렬 처리도 수준을 나누어 봐야 한다는 점이다. 서로 다른 문장을 배치로 묶어 동시에 계산하는 것과, 한 문장 안의 여러 위치를 동시에 계산하는 것은 다르다. RNN은 전자는 가능하지만 후자에서는 이전 hidden state를 기다려야 한다.

또 Attention이 이미 사용되고 있었다는 것과 Transformer가 제안한 변화도 구분하게 되었다. 기존에는 대부분 RNN으로 hidden state를 만든 뒤 Attention으로 필요한 정보를 참조했기 때문에 순차 계산의 제약이 남았다. Transformer는 위치 사이의 정보를 연결하는 구조 자체를 Attention 중심으로 바꿨다는 점이 다르다.

Self-Attention의 Self는 자기 위치만 본다는 뜻이 아니라 같은 시퀀스 안의 정보를 참조한다는 뜻이다. 그리고 정보 전달 경로가 일정하다는 말은 전체 계산량이 일정하다는 말과 다르다. 이 두 구분이 Background를 이해하는 데 중요했다.

남은 학습 과제는 Q·K·V의 행렬 차원을 직접 따라가고, decoder의 입력 이동과 mask가 정답을 미리 보는 것을 어떻게 막는지 작은 예제로 확인하는 것이다. 수식과 구조를 정리하는 것에서 더 나아가 각 단계의 계산을 설명할 수 있도록 이해할 필요가 있다.